機械学習演習

第1回：ガイダンス・環境構築・基礎知識

計良 宥志 ・ 千葉大学

2026年度

本日の内容

1. **授業概要** — シラバス・成績評価・進め方
2. **計算環境の設定** — エディタ・Git・conda
3. **演習に備えた基礎知識** — 線形代数と最適化の復習
4. **機械学習の勉強のコツ** — 概念の分解と理解の階層

1. 授業概要

授業の位置づけ

本授業は，同時に開講される講義「機械学習I」に対応した演習を行う．
「機械学習I」で学ぶ各項目に対して，**発展的内容や別視点での説明**を与え，
また**実装を通して実践的な習熟**を目指す．

- 講義「機械学習I」で理論を学ぶ → **本演習で手を動かして理解を深める**
- 全15回・毎回が講義に対応

到達目標

1. 「機械学習I」で学ぶ基本的な機械学習の理解を深める
   発展的内容や別視点での説明に触れ，**広い視座での理解**を得る
2. 基本的な機械学習の**アルゴリズム等を実装**できる
3. 学習理論・最適化理論が示す**理論的結果を実験により検証**できる

教員情報・成績評価・注意事項

教員

- **計良 宥志** (kera@chiba-u.jp)
- 自然科学系総合研究棟2 301
- 専門：深層学習，コンピュータビジョン

成績評価

- **Moodle** 上での小テスト／実装の提出

注意事項

- **自身のPC・電源アダプター**を持参すること
- 前回の機械学習Iの内容を**復習**すること

授業外学習

- 「機械学習I」の復習
- Python等の基本的なプログラミングの自習

2. 計算環境の設定

概要：セットアップする3つのもの

#	ツール	用途
1	**Cursor** または **VS Code + GitHub Copilot**	コードエディタ + AI補助
2	**Git / GitHub**	バージョン管理・コード共有
3	**conda** (Anaconda)	Python仮想環境の管理

> いずれも**学生は無料で上位機能**が使える．各自で学生認証を申請すること．

Cursor / VS Code + GitHub Copilot

できること

- **コード補完** — 続きを予測して自動入力する
- **チャットで質問** — エラーの原因や書き方をその場で問い合わせられる
- **コードの説明・リファクタリング** — 選択範囲の説明や書き換えを依頼できる
- **ターミナル統合** — エディタ内でコマンドを実行できる
- **Git統合** — 差分確認・コミットをGUIで操作できる
- **拡張機能** — Jupyter Notebook, Python, LaTeX などの豊富なプラグインがある

> Cursorは VS Code ベースのAI特化エディタである．GitHub Copilot は VS Code の拡張機能として導入する．
> **どちらでもよい**（慣れているほうでよい）．

Git / GitHub：なぜバージョン管理が必要か？

次のような経験はないだろうか

- `report_final.py`，`report_final2.py`，`report_本当のfinal.py` ……といったファイル名の乱立
- 「動いてたコードをいじったら壊れた．元に戻せない」
- 「チームで同じファイルを編集したら上書きされた」
- 「先週のバージョンに戻したいけど，どれだっけ？」
- 「このコード誰が何のために変えたんだっけ？」

> **Git** はこれらをすべて解決する「タイムマシン付きの変更記録係」である．

Git の基本的な使い方

```
# リポジトリの作成
git init

# 変更をステージング → コミット（記録）
git add ファイル名
git commit -m "変更内容のメッセージ"

# 変更履歴を確認
git log --oneline

# リモート（GitHub）にプッシュ
git push origin main
```

GitHub にリポジトリを作成し，自分の演習コードを管理すること.

conda・仮想環境：なぜ必要か？

次のような経験はないだろうか

- 「`pip install` したらPCの別のプログラムが動かなくなった」
- 「Aの課題では numpy 1.x，Bの課題では numpy 2.x が必要…」
- 「友達のコードを自分のPCで実行するとエラーになる」
- 「先月動いたスクリプトが今日はなぜか動かない」
- 「OSのPythonを壊してしまって色々大変なことに…」

> 仮想環境とは，**プロジェクトごとに独立したPythonの部屋**を作る仕組みである．

Anaconda のインストール

1. anaconda.com からインストーラをダウンロード
2. 自分のOSに合ったものを選択（Windows / macOS / Linux）
3. インストーラの指示に従いインストール
4. ターミナル（macOS/Linux）またはAnaconda Prompt（Windows）を開く

```
# インストール確認
conda --version
```

conda による仮想環境の作り方・使い方

```
# 環境の作成（Python 3.11 を指定）
conda create -n ml-exercise python=3.11

# 環境の有効化
conda activate ml-exercise

# 依存パッケージのインストール（pip）
pip install -r requirements.txt

# 環境の一覧確認
conda env list

# 環境の無効化
conda deactivate
```

本演習では `requirements.txt` を配布する．仮想環境を有効化したうえで `pip install -r requirements.txt` とすると，同一の依存関係を再現できる．
NumPy / SciPy / scikit-learn などは **pip で問題なくインストール**できる．

3. 演習に備えた基礎知識

この演習で使う基礎知識

機械学習を実装・理解するために，**Python** と **線形代数・最適化** の基礎が必要である.

機械学習の三要素

比喩	実体	説明
教材に相当するもの	**データセット**	モデルが学ぶための材料
学習する主体に相当するもの	**AIモデル**	データからパターンを学ぶ主体
指導に相当するもの	**最適化**	モデルを改善する仕組み

- **AIモデルの理解** ← 線形代数（計算のフローを追う）
- **最適化の理解** ← 微分・最適化理論

線形代数が必要な例 (1)：MLPの計算フロー

Multi-Layer Perceptron（多層パーセプトロン）

入力 $x \in \mathbb{R}^d$ に対して：

$$h_1 = \sigma(W_1 x + b_1), \quad h_2 = \sigma(W_2 h_1 + b_2), \quad \hat{y} = W_3 h_2 + b_3$$

- 各層は **行列の積** → **バイアス加算** → **活性化関数** の繰り返し
- $W_1 \in \mathbb{R}^{m_1 \times d}$, $W_2 \in \mathbb{R}^{m_2 \times m_1}$, $W_3 \in \mathbb{R}^{o \times m_2}$
- $b_1 \in \mathbb{R}^{m_1}$, $b_2 \in \mathbb{R}^{m_2}$, $b_3 \in \mathbb{R}^{o}$, $\sigma : \mathbb{R} \to \mathbb{R}$

> 線形代数が分かると，ニューラルネットワークの計算が「行列の連鎖」に見える.

線形代数が必要な例 (2)：Attention の計算フロー

Attention（注意機構）は，現在のAI（Transformer）の中核である．

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $Q \in \mathbb{R}^{N_q \times d_k}$, $K \in \mathbb{R}^{N_k \times d_k}$, $V \in \mathbb{R}^{N_k \times d_v}$
- $QK^\top$：全てのQuery–Keyペアの類似度を**行列積**で一括計算
- softmax で正規化 → V と加重和

> どんなに大きなモデルでも，計算フローは線形代数の組み合わせである．

最適化の基礎 (1)：スカラー関数のベクトル微分

スカラー値関数 $f : \mathbb{R}^n \to \mathbb{R}$ の勾配を
$\nabla f(x) \in \mathbb{R}^n$ と書く.

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix} \in \mathbb{R}^n$$

演習：ベクトル微分（3問）

問1

$x, c \in \mathbb{R}^n$ とする.

$$f_1(x) = c^\top x$$

$\nabla f_1(x)$ を求めよ.

計算欄：

問2

$x \in \mathbb{R}^n$ とする.

$$f_2(x) = \|x\|_2^2 = x^\top x$$

$\nabla f_2(x)$ を求めよ.

計算欄：

問3

$x \in \mathbb{R}^n$, $M \in \mathbb{R}^{n \times n}$（対称）とする.

$$f_3(x) = x^\top M x$$

$\nabla f_3(x)$ を求めよ.

最適化の基礎 (2)：Lagrange の未定乗数法

設定

等式制約付きの最適化問題

$$\max_{x \in \mathbb{R}^n} f(x) \quad \text{s.t.} \quad g(x) = 0, \quad g : \mathbb{R}^n \to \mathbb{R}$$

Lagrangian を導入：

$$\mathcal{L}(x, \lambda) = f(x) - \lambda g(x), \quad (x, \lambda) \in \mathbb{R}^n \times \mathbb{R}$$

必要条件（停留条件）：

$$\nabla_x \mathcal{L}(x, \lambda) = 0, \quad \partial_\lambda \mathcal{L}(x, \lambda) = 0$$

Lagrange の未定乗数法：例題

問題設定

$A \in \mathbb{R}^{n \times n}$ を対称行列とし，次を解け：

$$\max_{x \in \mathbb{R}^n} x^\top A x \quad \text{s.t.} \quad \|x\|_2 = 1$$

作業指示

1. 制約を $g(x) = x^\top x - 1 = 0$ と置く.
2. $\mathcal{L}(x, \lambda) = x^\top A x - \lambda(x^\top x - 1)$ を作る.
3. $\nabla_x \mathcal{L}(x, \lambda) = 0,\ \partial_\lambda \mathcal{L}(x, \lambda) = 0$ を立てる.
4. 得られる方程式の意味を考察する.

計算欄：

Lagrange 条件の幾何学的解釈（厳密寄り）

制約集合を

$$\mathcal{M} = \{x \in \mathbb{R}^n \mid g(x) = 0\}$$

とし，$x^\star \in \mathcal{M}$ を局所最適解とする.
$\nabla g(x^\star) \neq 0$ なら，接空間は

$$T_{x^\star}\mathcal{M} = \{v \in \mathbb{R}^n \mid \nabla g(x^\star)^\top v = 0\}$$

である．局所最適性より任意の接ベクトル $v \in T_{x^\star}\mathcal{M}$ に対して

$$\nabla f(x^\star)^\top v = 0$$

が必要である．したがって $\nabla f(x^\star)$ は接空間に直交し，
法線ベクトル $\nabla g(x^\star)$ と同一直線上にある：

$$\nabla f(x^\star) = \lambda \nabla g(x^\star), \quad \lambda \in \mathbb{R}.$$

図（2次元の概念図）

4. 機械学習の勉強のコツ

概念を分解して理解する

機械学習では多くの用語・概念が登場する.
闇雲にすべてを理解しようとするのではなく，**以下の階層に分解**すると整理しやすい.

1. **どのレイヤーの概念か**（上位概念？下位概念？）
2. **何のためのものか**（何をするのに便利？）
3. **入力と出力は何か**
4. （制約と条件は何か）
5. **そのアイディアは何か**
6. その仕組み・計算の流れ
7. その原理・理解

> **この順序で，どこかで理解を止める**．すべてを理解しようとしないことが重要である.

演習：概念の分解（例0）— 微分

対象：**スカラー関数の微分**（例：$f : \mathbb{R} \to \mathbb{R}$ や $f : \mathbb{R}^n \to \mathbb{R}$ の偏微分）

次の項目ごとに，一言でよいので自分の言葉で埋めること.

1. **どのレイヤーの概念か**（上位・下位の位置づけ）
2. **何のためのものか**
3. **入力と出力は何か**（型や記号も含めて）
4. **制約と条件は何か**（該当しなければ「なし」）
5. **そのアイディアは何か**
6. **仕組み・計算の流れ**（どこまで書けるか）
7. **原理・理解**（ここまで辿りつかなくてよい）

記述欄：

演習：概念の分解（例1）— 固有値（分解）

対象：**正方行列 $A \in \mathbb{R}^{n \times n}$ の固有値・固有ベクトル**（固有値分解のイメージまで）

同上の7項目に沿って記述すること.

1. どのレイヤーの概念か
2. 何のためのものか
3. 入力と出力は何か（A, λ, v の関係を明示）
4. 制約と条件は何か（例：$v \neq 0$ など）
5. そのアイディアは何か
6. 仕組み・計算の流れ
7. 原理・理解

記述欄：

演習：概念の分解（例2）— Lagrange の未定乗数法

対象：**等式制約付き最適化**

$$\max_{x \in \mathbb{R}^n} f(x) \text{ s.t. } g(x) = 0,\ \ g : \mathbb{R}^n \to \mathbb{R}$$

同上の7項目に沿って記述すること（$\mathcal{L}(x, \lambda)$ や停留条件に触れてもよい）.

1. どのレイヤーの概念か
2. 何のためのものか
3. 入力と出力は何か（f, g, x, λ など）
4. 制約と条件は何か（正則性条件など，知っていれば）
5. そのアイディアは何か
6. 仕組み・計算の流れ
7. 原理・理解（幾何学的イメージがあれば）

記述欄：

高校までと大学での勉強法の違い

問題になりやすい進め方

1. 仕組み・計算の流れから始める
2. 原理・理解に進む
3. 入力・出力や条件は問題文に書いてあるからスキップする

→ 高校までは問題が限定的なので通用した

大学・研究で望ましい進め方

1. まず「何のためのものか」を掴む
2. 入力と出力，条件を把握する
3. アイディアを理解する
4. 必要なら仕組み・原理へ進む

→ 多くの道具の中から**どれを使うか判断**する力が必要である

> 大学では「解き方を覚える」より「**どの道具をいつ使うか**」が重要である.

まとめ

- **授業概要**：「機械学習I」の演習として，発展的内容の理解と実装を行う
- **計算環境**：Cursor（または VS Code），Git/GitHub，conda を本日中にセットアップする
- **基礎知識**：線形代数（行列演算）と最適化（微分・Lagrange）が基盤
- **勉強のコツ**：概念を階層的に分解し，上位から順に理解する

> 次回は「機械学習I」第1回の内容に対応した演習を行う.
> 次回までに環境構築を完了させること.

以上

質問のある者は挙手すること.